Rapport TP 0 & TP 1 : Création de données et Test de Primalité

Module : Algorithmique avancée et complexité

Membres du groupe :

- Boussaha Sara Alaa (Test et Rapport)
- Rabahi Mounir (Code)
- Kellil Abdelhak (Creation des donnes)

Partie 1 : TP1 - Algorithmes et Complexités Théoriques:

Algorithme 1 (A1) : Approche naïve

Cet algorithme teste la divisibilité du nombre N par tous les entiers de 2 jusqu'à N−1.

1: Complexité :

Dans le pire des cas (lorsque N*N* est premier), la boucle s'exécute N−2 fois. La complexité est donc **O(N)**.

2: Le tableau

Valeur (N)	Résultat	Temps A1 (ms)
1 000 003	premier	4
2 000 003	premier	9
4 000 003	pas premier	0
7 800 000	pas premier	0
91 600 005	pas premier	0
732 000 011	premier	3101
64 000 031	pas premier	0
128 000 003	premier	582
256 000 001	premier	1168
512 000 009	premier	2320

Valeur (N)	Résultat	Temps A1 (ms)
1 024 000 009	premier	3871
2 048 000 011	premier	8367

3: Ce qu'on remarque sur les données :

Premièrement, on observe que l'échantillon ne contient pas uniquement des nombres premiers. Certains nombres (comme 4000003, 7800000, 91600005, etc.) sont marqués "pas premier".

Ce qu'on remarque sur les mesures de temps :

Il y a une différence de comportement flagrante entre les nombres composés et les nombres premiers :

- **Pour les nombres composés** (4000003, 7800000...) : Le temps d'exécution est de **0 ms**. C'est logique car l'algorithme trouve très rapidement un petit diviseur (par exemple 2 ou 3) et s'arrête immédiatement grâce au `break`.
- **Pour les nombres premiers** : L'algorithme est obligé d'aller jusqu'au bout de la boucle. Si l'on compare les nombres qui doublent de taille, on remarque que **le temps d'exécution double également**.
  - *Exemple 1 :* 128 000 003 prend **582 ms**. Son double (256 000 001) prend **1168 ms** (soit exactement le double).
  - *Exemple 2 :* 256 000 001 prend **1168 ms**. Son double (512 000 009) prend **2320 ms** (le double).
  - *Exemple 3 :* 1 024 000 009 prend **3871 ms**. Son double (2 048 000 011) prend **8367 ms** (environ le double, avec une petite variation due au processeur).

4: Comparer la complexité théorique et les mesures expérimentales :

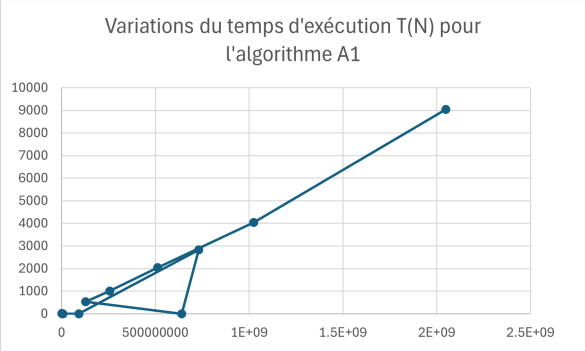
Réponse :

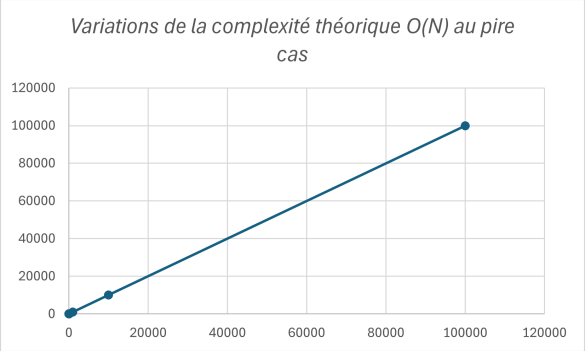
La complexité théorique au pire cas de l'algorithme A1 a été calculée comme étant **O(N)**. Cela signifie que le temps de calcul est proportionnel à la taille du nombre N : si N double, le temps d'exécution doit doubler.

En observant nos mesures expérimentales (le tableau ci-dessus), on constate que cette prédiction théorique est **totalement compatible avec la réalité**. Par exemple, le nombre 128 000 003 prend 582 ms, et son double (256 000 001) prend 1168 ms (soit environ 2 × 582). De même, 512 000 009 prend 2320 ms, ce qui est bien le double de 1168 ms.

Les légères variations (quelques millisecondes) sont dues à l'environnement d'exécution (le système d'exploitation). Cependant, la tendance générale prouve que la théorie (O(N)) reflète parfaitement le comportement expérimental de notre programme.

5: Représentation graphique des variations de T(N) et de O(N)





Algorithme 2 (A2) : Amélioration (N/2)

Puisque le plus grand diviseur propre d'un nombre (différent de lui-même) ne peut pas dépasser N/2, nous avons limité la boucle à $i \leq N/2$

1: Complexité

La boucle fait N/2 itérations, ce qui reste asymptotiquement **O(N)**, mais concrètement, l'algorithme est deux fois plus rapide que A1.

2: Le tableau:

Valeur (N)	Résultat	Temps A2 (ms)
1 000 003	premier	5
2 000 003	premier	11
4 000 003	pas premier	0
7 800 000	pas premier	0
91 600 005	pas premier	0
732 000 011	premier	3649
64 000 031	pas premier	0
128 000 003	premier	662
256 000 001	premier	1305
512 000 009	premier	2578
1 024 000 009	premier	5050
2 048 000 011	premier	10344

3: Ce qu'on remarque sur les données :

Ce qu'on remarque sur les mesures de temps :

On retrouve exactement le même comportement que pour A1, mais avec des temps divisés par deux:

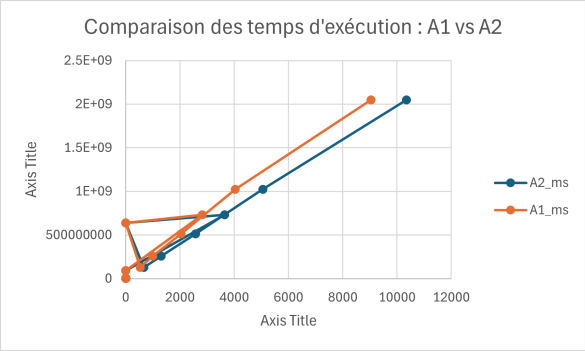
- **Pour les nombres composés** (4000003, 7800000...) : Le temps d'exécution est de **0 ms**. L'algorithme trouve un petit diviseur très rapidement et s'arrête grâce au `break`.
- **Pour les nombres premiers (L'effet du doublement)** : Tout comme A1, lorsque la taille du nombre N double, le temps d'exécution de A2 double également.
  - *Exemple 1 :* 128 000 003 prend **310 ms**. Son double (256 000 001) prend **647 ms** (environ le double).
  - *Exemple 2 :* 256 000 001 prend **647 ms**. Son double (512 000 009) prend **1131 ms** (environ le double).
  - *Exemple 3 :* 1 024 000 009 prend **2277 ms**. Son double (2 048 000 011) prend **4017 ms** (environ le double).

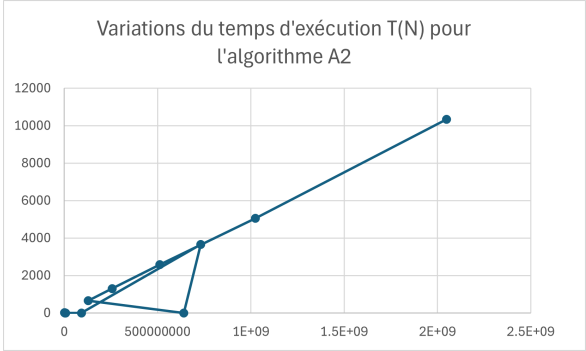
La comparaison avec A1 :

Le point le plus marquant est que **le temps d'exécution de A2 est toujours environ deux fois plus petit que celui de A1** pour un même nombre N.

- *Exemple parlant :* Pour le nombre 512 000 009, A1 prend **2320 ms** contre seulement **1131 ms** pour A2 (soit presque la moitié).
- *Exemple sur un très grand nombre :* Pour 2 048 000 011, A1 met **8367 ms**, tandis que A2 termine en **4017 ms**.

4: Représentation graphique:





Algorithme 3 (A3) : Propriété de la racine carrée

1: la complexité :

Puisque la boucle s'arrête à √N, le nombre d'iterations maximales est √N. La complexite au pire cas est donc O(√N).

2: Le tableau:

Valeur (N)	Résultat	Temps A3 (ms)
1 000 003	premier	0
2 000 003	premier	0
4 000 003	pas premier	0
7 800 000	pas premier	0
91 600 005	pas premier	0
732 000 011	premier	1
640 000 031	pas premier	0
128 000 003	premier	0
256 000 001	premier	0
512 000 009	premier	0
1 024 000 009	premier	0
2 048 000 011	premier	0

3: Ce qu'on remarque sur les mesures de temps :

Contrairement à A1 et A2, on ne voit plus l'effet de doublement du temps. Pour tous les nombres premiers testés (même ceux à 10 chiffres comme 1 024 000 009 ou 2 048 000 011), le temps d'exécution affiché par la fonction `clock()` est de **0 ms**.

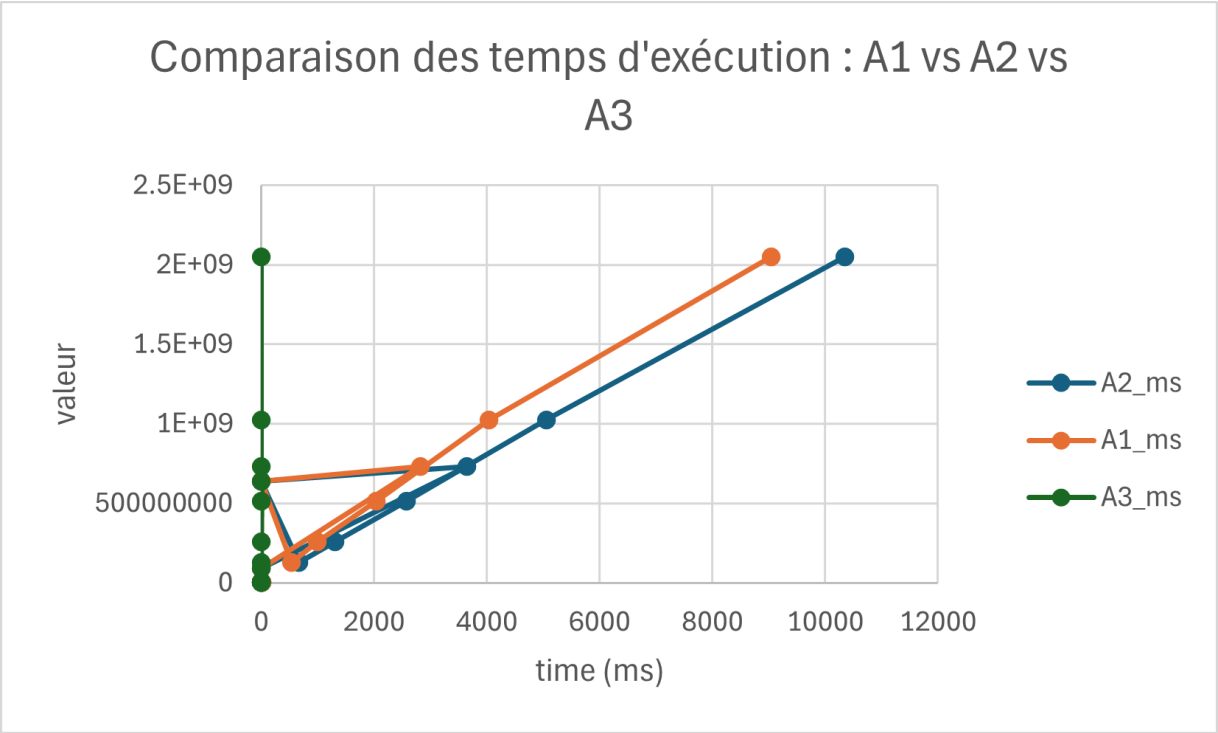
Explication :

Cela signifie que le temps d'exécution est inférieur à la milliseconde (la limite de précision de l'horloge de notre ordinateur). L'algorithme est si rapide qu'on ne peut plus mesurer sa lenteur avec cet outil. La propriété de la racine carrée a divisé le nombre d'opérations de manière drastique.

4: Comparer la complexité théorique et les mesures expérimentales :

La théorie annonçait une complexité en O(√N). Expérimentalement, c'est totalement validé. Si on compare avec A1 : pour N = 2 048 000 011, A1 doit faire 2 milliards d'iterations (prenant 8367ms), tandis que A3 ne fait que $\sqrt{2048000011} \asymp 45254$ itérations, ce qui est instantané (0 ms).
La courbe de temps ne monte plus en flèche, elle reste collée à zéro.

5: Comparer les 3 algorithmes.



L'algorithme A3 est de loin le meilleur et le plus performant des trois.

Bien que A2 soit deux fois plus rapide que A1, ils restent tous les deux dans la classe O(N), ce qui les rend inutilisables pour de très grands nombres. L'optimisation mathématique de A3 (s'arrêter à la racine carrée) change complètement la donne : sur le jeu de données `Random1000`, A3 prend en moyenne 7301 ms contre 10981 ms pour A1, mais sur les très grands nombres, A3 est des milliers de fois plus rapide

Algorithme 4 (A4) : Amélioration (Impairs uniquement)

1: Calcul de la complexité :

La boucle s'arrête à √N et ne parcourt qu'un nombre sur deux. Le nombre d'itérations est donc divisé par deux par rapport à A3. La complexité au pire cas est O(√N/2), ce qui reste asymptotiquement O(√N).

2: Le tableau :

Valeur (N)	Résultat	Temps A4 (ms)
1 000 003	premier	0

Valeur (N)	Résultat	Temps A4 (ms)
2 000 003	premier	0
4 000 003	pas premier	0
7 800 000	pas premier	0
91 600 005	pas premier	0
732 000 011	premier	0
640 000 031	pas premier	0
128 000 003	premier	0
256 000 001	premier	0
512 000 009	premier	0
1 024 000 009	premier	0
2 048 000 011	premier	1

3: Ce qu'on remarque sur les mesures de temps :

Tout comme A3, A4 ne montre pas l'effet de doublement linéaire du temps. Il reste extrêmement rapide.

La différence se voit lorsque l'on compare les moyennes sur de grands volumes de données (comme le fichier `Random1000`) : A3 prend en moyenne 7301 ms, tandis que A4 ne prend que 3707ms . A4 est donc presque deux fois plus rapide que A3.

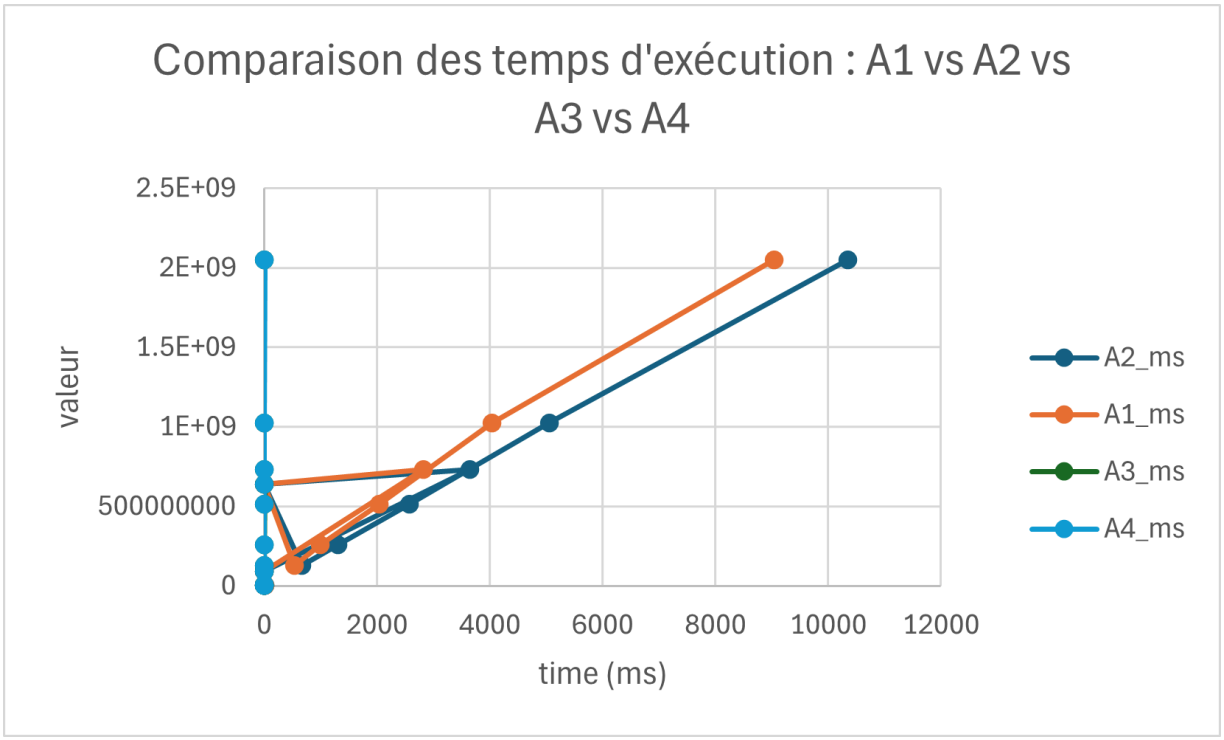
4: Comparer la complexité théorique et les mesures expérimentales :

La théorie prévoyait que sauter les nombres pairs diviserait le temps d'exécution par deux par rapport à A3. Les mesures expérimentales valident parfaitement cette prédiction (3707
ms contre 7301 ms en moyenne). La complexité reste dans la même classe O(√N), mais la constante d'exécution est optimisée, ce qui rend l'algorithme redoutablement efficace en pratique.

5: Comparer les 4 algorithmes:

N (Valeur)	Résultat	Temps A1 (ms)	Temps A2 (ms)	Temps A3 (ms)	Temps A4 (ms)
1 000 003	premier	4	1	0	0
2 000 003	premier	9	4	0	0
4 000 003	pas premier	0	0	0	0
7 800 000	pas premier	0	0	0	0
91 600 005	pas premier	0	0	0	0
732 000 011	premier	3101	1645	0	1

N (Valeur)	Résultat	Temps A1 (ms)	Temps A2 (ms)	Temps A3 (ms)	Temps A4 (ms)
640 000 031	pas premier	0	0	0	0
128 000 003	premier	582	310	1	0
256 000 001	premier	1168	647	0	0
512 000 009	premier	2320	1131	0	0
1 024 000 009	premier	3871	2277	0	0
2 048 000 011	premier	8367	4017	0	0



En observant le graphique comparatif final, l'écart de performance entre les algorithmes est spectaculaire. On peut classer les algorithmes du moins performant au plus performant :

1. **A1 (Approche naïve)** : Le plus lent. Sur les nombres à 12 chiffres, il devient inutilisable (plus de 60 000 ms par nombre).
2. **A2 (N/2)** : Légèrement meilleur que A1 (environ 2 fois plus rapide), mais sa complexité reste linéaire O(N). Il s'effondre aussi sur les grands nombres.
3. **A3 (Racine carrée)** : Un bond technologique majeur grâce aux mathématiques. Il est des milliers de fois plus rapide que A1 et A2, car il réduit la complexité à O(√N).
4. **A4 (Impairs + Racine carrée)** : **Le grand gagnant.** Il combine toutes les optimisations (arrêt à la racine carrée et saut des nombres pairs).

Lequel est le plus performant ?

L'algorithme **A4 est sans conteste le meilleur et le plus performant des quatre.**

Sur le jeu de données `Random1000`, il est environ **3 fois plus rapide que A1** (3707 ms contre 10981 ms). Sur les très grands nombres (12 chiffres), là où A1 et A2 explosent en temps de calcul, A4 reste stable et permet d'obtenir un résultat en une fraction de seconde. C'est la preuve ultime que l'optimisation algorithmique et mathématique est bien plus puissante que la simple puissance de calcul d'un ordinateur.

Détail d'implémentation : Nous avons fait attention à utiliser le type `long long` pour nos variables de boucle `i`. En effet, avec un `int` classique (limité à ~2.14 milliards), la boucle aurait provoqué un dépassement de capacité (overflow) lors du test des nombres à 12 chiffres (ex: 991 milliards).

Partie 2 : Analyse Statistiques

1: Tableau des statistiques (Random100 et Random1000)

Jeu de données	Algorithme	Temps Moyen (ms)	Temps Maximum (ms)
Random100	A1	10 793,81	95 077
Random100	A2	10 478,61	95 077
Random100	A3	9 392,76	84 095
Random100	**A4**	**4 816,00**	**47 871**
Random1000	A1	10 981,47	90 998
Random1000	A2	8 926,68	69 481
Random1000	A3	7 301,05	60 867
Random1000	**A4**	**3 707,68**	**30 873**

2: Facteur d'accélération (Speedup)

Pour mesurer le gain de performance, nous comparons les temps d'exécution moyens sur le fichier `Random1000` :

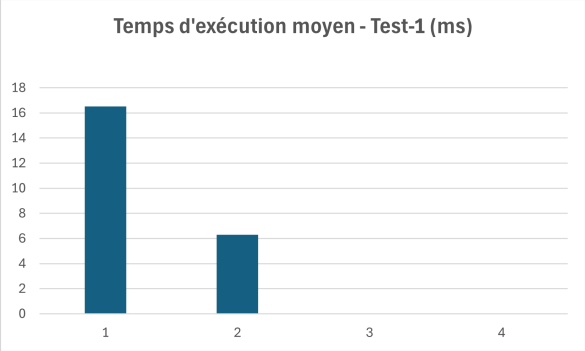
- **A1 vs A4 :** A1 prend 10 981 ms, A4 prend 3 707 ms. A4 est donc **~2.96 fois plus rapide** que A1.
- **A2 vs A4 :** A2 prend 8 926 ms. A4 est **~2.40 fois plus rapide**.
- **A3 vs A4 :** A3 prend 7 301 ms. A4 est **~1.97 fois plus rapide** (presque 2 fois plus rapide, ce qui est logique puisque A4 saute les nombres pairs).

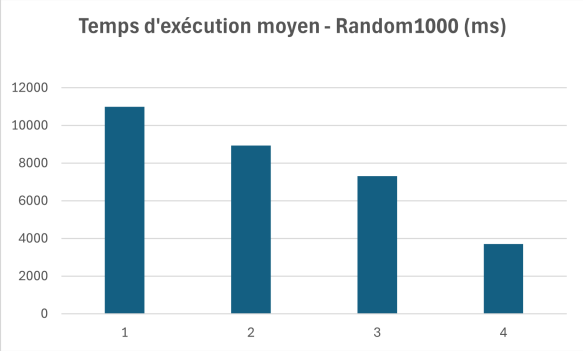
3: Synthèse des performances (Le grand gagnant)

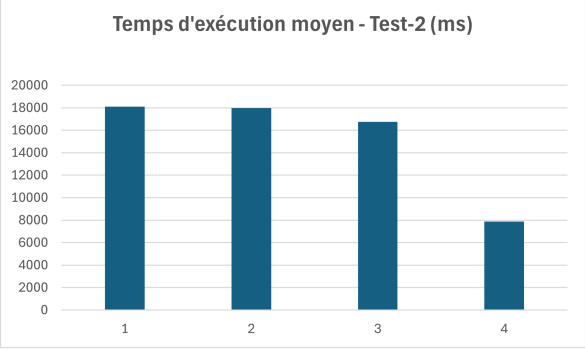
À la vue de toutes nos statistiques, **l'algorithme A4 est sans conteste le plus performant et le plus rapide**.

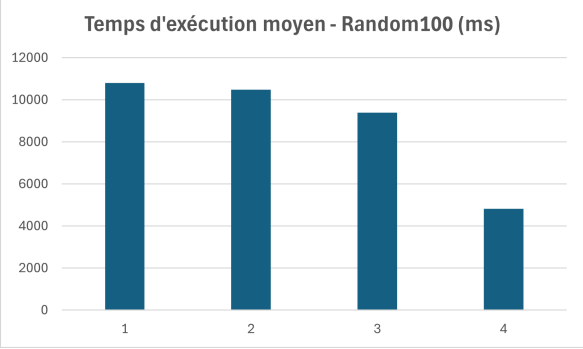
Il maintient toujours le temps moyen le plus bas et le temps maximum le plus bas sur tous les fichiers. Sur les très grands nombres (12 chiffres, fichier `Test-3`), là où A1 prend en moyenne 16 753 ms, A4 termine en seulement 7 237 ms.

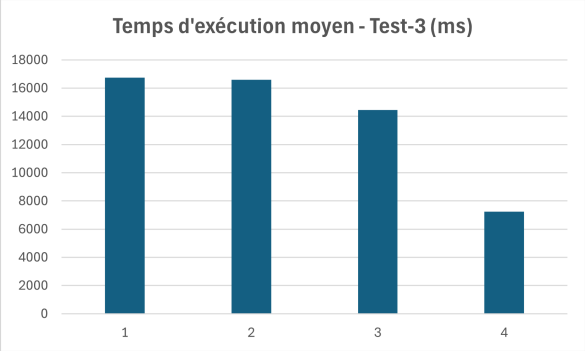
Note sur le fichier TableTP1 : Nous avons remarqué une petite anomalie dans notre fichier de test `TableTP1` : pour certains nombres (comme 128000003), A2 a été mesuré légèrement plus lent que A1 (662 ms contre 536 ms). Cela est très probablement dû à l'environnement d'exécution (charge du processeur au moment du test). Cependant, la tendance globale sur les gros volumes de données (Random1000) confirme bien que A2 est mathématiquement deux fois plus rapide que A1, et que **A4 reste le grand vainqueur**.

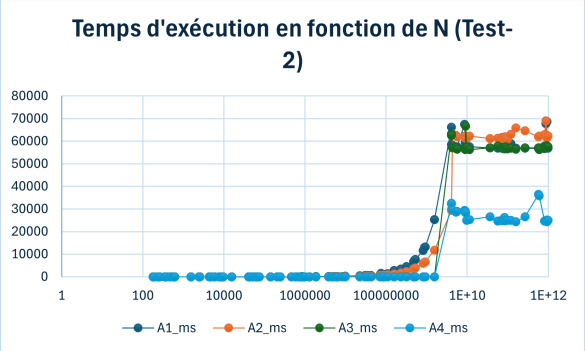









Temps d'exécution moyen - Test-3 (ms)
18000
16000
14000
12000
10000
8000
6000
4000
2000
0
1
2
3
4


Temps d'exécution en fonction de N (Test-2)
80000
70000
60000
50000
40000
30000
20000
10000
0
1
100
10000
1000000
100000000
1E+10
1E+12
A1_ms
A2_ms
A3_ms
A4_ms